Adam (Chiyong Seong)

AI Engineer & Automation Consultant

Email: giha1205@gmail.com
GitHub: github.com/giha-create
Location: Seongnam, South Korea

핵심 역량

LLM Agent 개발

Gemini, Claude API 기반 멀티 에이전트 시스템 설계 및 구현. LangGraph 워크플로 자동화.

업무 프로세스 자동화

리서치, 이메일 작성, 리드 스코어링 등 반복 업무를 AI로 자동화하여 40% 이상 시간 절감.

실시간 데이터 파이프라인

주식/코인 시세, 뉴스, 소셜 데이터를 실시간 수집하여 자동 분석 및 의사 결정 지원.

클라우드 배포 & DevOps

AWS EC2 + Docker Compose 기반 프로덕션 배포. CI/CD 파이프라인 구축 경험.

기술 스택

Python · LangGraph · Gemini API · Claude API · Telegram Bot · Flask · Docker · AWS EC2 · SQLite · SendGrid · Chart.js · GitHub Actions

프로젝트 포트폴리오

AI Sales Automation

AI 기반 B2B 영업 자동화 시스템 (2025~현재, 실서비스 운영중)

LangGraph 멀티 에이전트가 기업 리서치 → 리드 스코어링 → 맞춤 이메일 작성 → 승인 → 발송까지 전 과정을 자동화. Epsilon-Greedy A/B 테스팅으로 이메일 템플릿을 지속 최적화하며, 실시간 대시보드로 성과 모니터링. 텔레그램 HITL(Human-in-the-Loop) 인터페이스로 모바일에서 즉시 승인/거부 가능.

Python | LangGraph | Gemini API | SendGrid | Telegram Bot | Flask | Docker | AWS EC2

Jarvis AI Assistant

개인 비서 AI 에이전트 (2025, 실사용중)

Claude API 기반 개인 비서. 일정 관리, 이메일 요약, 웹 검색, 파일 관리 등 일상 업무를 자연어 명령으로 처리. 컨텍스트 유지 대화, 도구 호출(Tool Use), 멀티턴 추론 등 고급 에이전트 패턴 적용.

Python | Claude API | Tool Use | Async | SQLite

AI Trading System

AI 주식/코인 자동매매 시스템 (2025, 실계좌 운영중)

실시간 시세 데이터 수집 → AI 기술적 분석 → 자동 매수/매도 실행. 한국투자증권 API 및 업비트 API 연동. Gemini AI가 차트 패턴, 거래량, 뉴스 센티먼트를 종합 분석하여 매매 신호 생성. 텔레그램으로 매매 알림 및 수익률 리포트 자동 전송.

Python | Gemini API | 한국투자증권 API | Upbit API | Telegram | SQLite

AI Agent Telegram

텔레그램 Gemini & Claude AI 에이전트 (2025, 실사용중)

텔레그램 메신저 내에서 Gemini/Claude 듀얼 AI와 대화. 이미지 분석, 코드 생성, 문서 요약, 번역 등 다목적 AI 비서. 사용자별 대화 히스토리 관리, 컨텍스트 기반 응답, 스트리밍 출력 지원.

Python | Gemini API | Claude API | python-telegram-bot | Async

Daily News Automation

매일 아침 AI 뉴스 브리핑 자동 전송 (2025, 매일 운영중)

매일 아침 7시, AI가 주요 뉴스를 자동 수집 → 요약 → 카테고리 분류 → 텔레그램 전송. 관심 분야(AI/테크/경제) 맞춤 필터링, 한줄 요약 + 상세 분석 2단계 제공.

Python | Gemini API | News API | Telegram Bot | Cron Scheduler

YouTube Shorts Automation

AI 유튜브 쇼츠 자동 생성 시스템 (2025)

트렌딩 주제 분석 → AI 스크립트 생성 → TTS 음성 합성 → 영상 편집 → 자동 업로드. 콘텐츠 기획부터 게시까지 전 과정을 파이프라인으로 자동화.

Python | Gemini API | TTS | FFmpeg | YouTube Data API

Nail Shop Budget

소상공인 네일샵 4년 재무 관리 웹 대시보드 (2025~2028, 실사용중)

소상공인 네일샵의 4년치 매출·비용·순수익을 월 단위로 기록·분석하는 Streamlit 웹 앱. Google Sheets 를 1차 저장소로, 실패 시 로컬 JSON 으로 자동 폴백하는 Storage 어댑터 + Dual-Write 패턴으로 데이터 내구성 확보. 로컬(.env+service_account.json) 과 프로덕션(st.secrets) 설정 소스를 한 코드로 흡수하는 Config Source Adapter 구현. 연도별 실제 납부 세금을 직접 차감해 세전·세후 실시간 비교, 모바일 우선 UI, Streamlit Community Cloud 에 배포 및 핸드폰 실사용 중.

Python | Streamlit | Google Sheets API | gspread | Plotly | pandas | Storage Adapter | Config Source Adapter | Streamlit Cloud

서비스 소개

제공 서비스

1. AI 기반 영업/마케팅 자동화

리드 발굴 → 기업 리서치 → 맞춤 이메일 발송까지 B2B 영업 프로세스를 AI로 자동화합니다.

2. 업무 프로세스 자동화 컨설팅

반복적인 데이터 수집, 보고서 작성, 알림 시스템 등을 AI 에이전트로 대체하여 비용을 절감합니다.

3. 맞춤형 AI 에이전트 개발

기업 특화 AI 챗봇, 데이터 분석 봇, 고객 응대 자동화 등 맞춤형 에이전트를 개발합니다.

4. AI 트레이딩 & 데이터 파이프라인

실시간 데이터 수집 → AI 분석 → 자동 실행 파이프라인을 구축합니다.

진행 프로세스

STEP 1	STEP 2	STEP 3	STEP 4
니즈 파악	솔루션 설계	개발 & 테스트	배포 & 운영지원

계약 방식

프리랜서 계약 (3.3% 원천징수) — 프로젝트 단위 계약. 소규모 자동화, 단기 컨설팅에 적합합니다.
간이사업자 계약 — 지속적 운영/유지보수가 필요한 경우. 세금계산서 발행 가능합니다.

모든 프로젝트는 **실제 운영 중인 시스템**을 기반으로 하며,
구축 후에도 지속적인 기술 지원을 제공합니다.

APPENDIX — Code & Architecture

GitHub: github.com/Adam-1228 | Production-level implementations

5	4,000+	200+	3
Active Projects	Lines of Code	Git Commits	Cloud Deployed

ai_sales_automation / graph / workflow.py

LangGraph | Production

```python
# LangGraph 워크플로우 — 순환형 검증 루프 + HITL + State DB 통합
# [리드] → research → write_email → review —(통과)→ hitl → send
#                ↑             |                    |
#                +—(미달)——+                    +—(재작성)→ write
from langgraph.graph import StateGraph, END
from agents.researcher import ResearchAgent
from agents.email_writer import EmailWriterAgent
from agents.reviewer import ReviewerAgent

class PipelineState(TypedDict):
    lead_id: str
    company_name: str
    research_data: str       # JSON string
    email_subject: str
    review_score: float
    review_passed: bool
    hitl_status: str          # pending / approved / rejected / timeout
    retry_count: int
```

★ **핵심 구현:** LangGraph StateGraph 기반 순환형 검증 루프 — AI 이메일 자동 검수, 미달 시 재작성 (max 3회), 통과 시 텔레그램 HITL 승인 대기 후 최종 발송

ai_sales_automation / agents / email_verifier.py

Security

```python
class EmailVerifier:
    DISPOSABLE_DOMAINS = {"guerrillamail.com", "tempmail.com", ...}
    ROLE_PREFIXES = {"info", "admin", "support", "contact", ...}

    async def verify(self, email: str) -> dict:
        # Stage 1: 형식 검증 (Regex)
        if not self._check_format(email): return {...}
        # Stage 2: MX 레코드 DNS 조회
        mx_valid = await self._check_mx_record(domain)
        # Stage 3: SMTP RCPT TO 핸드셰이크
        smtp_ok = await self._smtp_verify(email, mx_host)
        # Stage 4: Hunter.io API 교차검증
        return {"valid": True, "score": 95, "method": "smtp+hunter"}
```

★ **핵심 구현:** Regex → MX DNS → SMTP RCPT TO → Hunter.io 4단계 검증 + catch-all 탐지 + 일회용 메일 차단 — 발송 전 자동 검증으로 바운스 방지

Jarvis-AI-Assistant / server / server.py

Gemini Live | Claude API

```python
class JarvisBrain:
    # 자비스의 두뇌 - Gemini와 Claude를 연결하는 중앙 처리 장치
    def __init__(self):
        self.gemini_model = genai.GenerativeModel('gemini-2.0-flash')

    async def process_voice_command(self, command, language="ko"):
        analysis_prompt = f"사용자가 AI 비서 '자비스'에게 명령:
        PC 제어 필요시 ACTION: / 일반 대화시 CHAT: 으로 시작"
        result = self.gemini_model.generate_content(analysis_prompt)
        if result.startswith("ACTION:"):
            return await self.execute_with_claude(action_detail)

    async def execute_with_claude(self, action_detail, command):
        # Claude Computer Use API → 로컬 PC 에이전트 WebSocket 전달
```

★ **핵심 구현:** Gemini Live 음성 → 의도 분류 (ACTION vs CHAT) → PC 제어 시 Claude Computer Use API 위임 — FastAPI WebSocket 양방향 통신

APPENDIX — Code & Architecture

Real-time Trading · News Automation · AI Lead Scoring

upbit_trading / modules / ai_analyzer.py

Gemini AI | 24/7 Trading

```
class AIAnalyzer:
    # Gemini AI 기반 실시간 매매 판단 엔진
    def __init__(self):
        self.model = genai.GenerativeModel('gemini-2.0-flash')

    def analyze(self, ticker, current_price, volume, indicators):
        prompt = f"당신은 암호화폐 전문 트레이더입니다.
    현재 가격: {current_price}원
    거래량: {volume} | RSI: {indicators['rsi']}
    볼린저밴드: {indicators['bollinger']}
        MACD: {indicators['macd']}
    반드시 다음 형식으로 응답:
    거래 판단: BUY / SELL / HOLD
    신뢰도: 0-100 | 투자 비중: 0-100%"
        result = self.model.generate_content(prompt)
        return self._parse_decision(result.text)
```

★ **핵심 구현:** 기술지표(RSI, Bollinger Band, MACD) + AI 분석 융합 — 12개 모듈 분리로 24시간 자동매매, Upbit API 연동 실전 운용

open_trading_self / modules / trader.py

Claude AI | KR Stock

```
# Claude AI 기반 한국 주식 자동매매 봇 — 한국투자증권 API 연동
# 106 commits | 뉴스/날씨 브리핑 + AI 분석 통합
class StockTrader:
    async def analyze_with_claude(self, stock_data, news_summary):
        response = client.messages.create(
            model="claude-sonnet-4-20250514",
            messages=[{"role": "user", "content": prompt}]
        )
    async def execute_trade(self, decision):
        # 한국투자증권 Open API로 실제 주문 실행
        if decision["action"] == "BUY":
            await self.kis_api.place_order("buy", ticker, qty)
```

★ **핵심 구현:** Claude AI 매매판단 + 한국투자증권 Open API 실전 주문 연동 — 뉴스 크롤링/날씨 데이터 통합 분석, GitHub Actions CI/CD 배포

send_today_news / main.py

GitHub Actions | RSS + AI

```
NEWS_CATEGORIES = {
    "IT/기술": ["https://news.google.com/rss/...IT"],
    "경제":    ["https://news.google.com/rss/...경제"],
    "사회/세계": ["https://news.google.com/rss/...세계"],
}

def summarize_with_gemini(articles: list) -> str:
    # Gemini API로 뉴스 요약 (Fallback: Claude → 제목만)
    prompt = "다음 뉴스 기사들을 한국어로 간결하게 요약해주세요."
    try:
        return call_gemini(prompt)   # Primary
    except:
        return call_claude(prompt)   # Fallback

def send_to_telegram(summary, weather):
    # 텔레그램 봇으로 요약 뉴스 + 성남 날씨 전송
```

★ **핵심 구현:** Google RSS 수집 → Gemini/Claude 3단계 Fallback 요약 → 텔레그램 자동 전송 — GitHub Actions cron 매일 자동 실행, 성남 날씨 통합

ai_sales_automation / agents / lead_scorer.py

Gemini AI | Scoring

```
class LeadScorer:
    # Gemini AI 기반 리드 품질 자동 스코어링 (0-100점)
    async def score_and_save(self, lead_id, research_data, contact_title):
        prompt = f"다음 기업 리서치 데이터를 분석하여 리드 품질 점수를 매겨주세요.
    평가 기준: AI 도입 가능성, 기업 규모, 의사결정 직급, 기술 성숙도
    반드시 0-100 정수로만 응답."
        score = await self._call_gemini(prompt)
        await state_db.upsert_lead(lead_id, lead_score=score)
        return {"lead_id": lead_id, "score": score}
```

★ **핵심 구현:** 리서치 완료 직후 Gemini AI가 자동으로 리드 품질 평가 — AI 도입 가능성/기업 규모/직급 기반 0-100점 스코어링, 대시보드 분포 시각화

APPENDIX — Code & Architecture

Dashboard · Telegram HITL · A/B Testing · Infrastructure

AI Sales Dashboard — http://3.104.252.39:8050 (Live)

AWS EC2 | Flask

```python
# Flask + Chart.js 실시간 대시보드 — AWS EC2 Docker 라이브 배포
@app.route("/api/stats")
def get_stats():
    # 실시간 KPI: Total Leads, Emails Sent, Reply Rate, Avg Score
    return jsonify({
        "total_leads": 18, "emails_sent": 3,
        "avg_review_score": 84.5, "avg_lead_score": 27.9
    })
```

★ **핵심 구현:** Daily Trend, A/B Test 6개 변형 비교, Pipeline Status 도넛 차트, Lead Score 분포 — SQLite 실시간 조회, Docker Compose 배포

ai_sales_automation / bot / telegram_hitl.py

HITL | Telegram

```python
class TelegramHITL:
    # Human-in-the-Loop 텔레그램 봇 — 이메일 승인/거절 관리
    async def request_approval(self, lead_id, email_body, score):
        keyboard = [[
            InlineKeyboardButton("✔ 승인", callback_data=f"approve_{lead_id}"),
            InlineKeyboardButton("✘ 거절", callback_data=f"reject_{lead_id}"),
            InlineKeyboardButton("✎ 수정", callback_data=f"edit_{lead_id}"),
        ]]
        await bot.send_message(text=preview, reply_markup=markup)
        # 비동기 응답 대기 (asyncio.Event 기반 타임아웃)
        result = await asyncio.wait_for(self.events[lead_id].wait(), 3600)
```

★ **핵심 구현:** 텔레그램 InlineKeyboard 승인/거절/수정 — asyncio.Event 비동기 대기, /verify /replies /status 등 10개+ 명령어

ai_sales_automation / agents / ab_testing.py

Epsilon-Greedy | A/B Test

```python
# Epsilon-Greedy 알고리즘 기반 이메일 템플릿 자동 선택
VARIANT_MAP = {
    "cold_basic_v1":    "기본 콜드메일",
    "painpoint_v1":     "페인포인트 중심",
    "value_offer_v1":   "가치 제공형",
    "question_v1":      "질문형",
    "case_study_v1":    "사례 중심형",
    "insight_v1":       "인사이트형",
}

def select_template(epsilon=0.3):
    if random.random() > epsilon:
        return get_best_performing_template()
    return random.choice(list(VARIANT_MAP.keys()))
```

★ **핵심 구현:** 6개 이메일 템플릿 변형에 Epsilon-Greedy 알고리즘 적용 — 응답률 데이터 축적으로 자동 최적화, 대시보드에서 실시간 성과 비교

ai_sales_automation / db / state_manager.py

SQLite | State DB

```python
class StateDB:
    # 전체 파이프라인 상태 영속화 — SQLite 비동기 관리
    async def initialize(self):
        # 테이블 생성 + 마이그레이션 (멱등성 보장)
        migrations = [
            "ALTER TABLE leads ADD COLUMN lead_score INTEGER DEFAULT 0",
            "ALTER TABLE leads ADD COLUMN email_verified TEXT DEFAULT 'unverified'",
            "ALTER TABLE leads ADD COLUMN email_verify_score INTEGER DEFAULT 0",
        ]
        for sql in migrations:
            try: await db.execute(sql)
            except: pass  # 이미 컬럼 존재 시 무시 (멱등성)
```

★ **핵심 구현:** aiosqlite 비동기 DB 관리 + ALTER TABLE 멱등성 마이그레이션 패턴 — 파이프라인 전체 상태 영속화, 장애 복구 지원

APPENDIX — Landing Page & Pipeline

Full-Stack Deployment: Frontend + Serverless Backend + Analytics Dashboard

LP 랜딩페이지 개설 — Full-Stack 배포

반응형 랜딩 + Google Apps Script 서버리스 백엔드 + 실시간 대시보드 + Vercel CI/CD

HTML/CSS/JS | Google Apps Script | Google Sheets API | Vercel | Telegram Bot

시스템 아키텍처

```
사용자 → Vercel CDN → [랜딩페이지 HTML/CSS/JS] → fetch() POST
    ↓
[Google Apps Script 백엔드] → 입력 검증 + XSS 방어
    ↓                          ↓
[Google Sheets]              [Telegram Bot 알림]
 • 신청/방문/대시보드         • 신청 즉시 알림 + 보안 플래그
```

</>
반응형 랜딩
모바일 퍼스트 디자인. CSS Grid + Flexbox. 로딩 최적화.

λ
서버리스 백엔드
Google Apps Script. 서버 비용 $0. 자동 스케일링.

△
실시간 대시보드
전환율, UTM 추적, 직업별/금액별/시간대별 분석.

➢
텔레그램 알림
신청 즉시 알림. 보안 플래그 자동 탐지.

backend / api_handler.js — 서버리스 백엔드 Google Apps Script

```gas
const SHEET_SUBMISSIONS = 'submissions';
const SHEET_VISITS = 'visits';
const SHEET_DASHBOARD = 'dashboard';

function doPost(e) {
  const data = JSON.parse(e.postData.contents);
  // 입력 검증 + XSS 방어 + 보안 플래그 체크
  const sanitized = validateAndSanitize(data);
  if (!sanitized.valid) return error(sanitized.reason);

  // Google Sheets에 데이터 저장
  appendToSheet(SHEET_SUBMISSIONS, sanitized.data);
  // 텔레그램 실시간 알림
  sendTelegramNotification(sanitized.data);
  // 대시보드 KPI 자동 갱신
  updateDashboard();
}
```

★ **핵심 구현:** Google Apps Script 서버리스 백엔드 — 폼 데이터 수신 → 입력 검증/XSS 방어 → Sheets 저장 → 텔레그램 알림 → 대시보드 갱신, 서버 비용 $0

deploy / vercel.json — 보안 헤더 + CDN 배포 Vercel Security

```jsonc
{
  "headers": [{
    "source": "/(.*)",
    "headers": [
      { "key": "Content-Security-Policy",
        "value": "default-src 'self'; script-src 'self' 'unsafe-inline'..." },
      { "key": "Strict-Transport-Security",
        "value": "max-age=63072000; includeSubDomains; preload" },
      { "key": "X-Frame-Options", "value": "DENY" },
      { "key": "X-Content-Type-Options", "value": "nosniff" },
      { "key": "Permissions-Policy",
        "value": "camera=(), microphone=(), geolocation=()" }
    ]
  }]
}
```

★ **핵심 구현:** Vercel 배포 + 6개 보안 헤더 (CSP, HSTS, X-Frame-Options, XSS Protection, Permissions-Policy) — OWASP 보안 기준 준수

보안 + 대시보드 — 8개 보안 헤더 + 실시간 KPI Security Analytics

```gas
// 보안: CSP, HSTS, X-Frame-Options, XSS Protection, 입력 검증, Rate Limiting
// 대시보드: 전환율, UTM 추적, 직업별/금액별/시간대별/기기별 분석
function updateDashboard() {
  const dashboard = ss.getSheetByName(SHEET_DASHBOARD);
  dashboard.getRange("B2").setValue(totalVisits);
  dashboard.getRange("B3").setValue(totalSubmissions);
  dashboard.getRange("B4").setValue(conversionRate + "%");
}
```

★ **핵심 구현:** OWASP 기준 8개 보안 헤더 + Google Sheets 실시간 대시보드 (전환율/UTM/직업별/시간대별 분석, 5분 자동 갱신)

APPENDIX — Code & Architecture

Nail Shop Budget · Streamlit + Google Sheets · Mobile-First SaaS Deployment

budget_app / auth.py

Lightweight Gate | UX-First

```python
# 경량 게이트 — 도메인 유출 리스크 낮음 → 과도한 보안 제거
_PASS_PHRASE = "***"    # 사업자 지정 단어 (노출 시 크리티컬 X)

def render_login_form() -> bool:
    if st.session_state.get("authenticated"):
        return True
    with st.form("gate_form"):
        code = st.text_input("코드", type="password")
        if st.form_submit_button("입장"):
            if code.strip().lower() == _PASS_PHRASE:
                st.session_state["authenticated"] = True
                st.rerun()
    return False
```

★ **핵심 구현: 보안 레벨 = 위협 모델 × 비용**. 복잡한 쿠키·서명 체인은 이 도메인에 과보안 (노출 위험 = 금액 일부 뿐) → 단일 단어 게이트로 축소. **의도적 단순화 = 가장 어려운 엔지니어링 판단** 중 하나.

budget_app / data_store.py

Config Adapter | Dual-Write

```python
# Config Source Adapter — st.secrets (프로덕션) / .env (로컬) 이중 소스
def _try_build_sheets_client():
    sheet_id, has_secrets_creds = "", False
    try:                                    # 1) Streamlit Cloud
        if "GOOGLE_SHEET_ID" in st.secrets:
            sheet_id = str(st.secrets["GOOGLE_SHEET_ID"])
        has_secrets_creds = "gcp_service_account" in st.secrets
    except Exception: pass
    if not sheet_id:                        # 2) 로컬 .env fallback
        load_dotenv(); sheet_id = os.environ.get("GOOGLE_SHEET_ID", "")
    return SheetsClient(sheet_id) if sheet_id else None

class DataStore:                        # Dual-Write: 로컬 + Sheets 동시
    def save(self, df):
        local_save(df)                      # 항상 백업 (오프라인 내구성)
        if self._sheets: self._sheets.save(df)   # 원격 동기화
```

★ **핵심 구현: 같은 코드가 로컬과 프로덕션에서 모두 동작** — Twelve-Factor "Config in environment" 원칙의 실전. 로컬은 .env + JSON 파일, 배포는 `st.secrets` → 어댑터가 소스를 흡수. **Dual-Write** 로 Sheets 장애 시에도 로컬 JSON 으로 서비스 지속, 복구 후 다음 save 에서 자동 동기화.

budget_app / calculations.py

Pure Functions | Tax Logic

```python
# 순수 함수 + 연도별 실제 납부액 차감 (비율 X → 세무사 고지액 기준)
def apply_tax(net_before: float, tax_amount: float) -> dict:
    return {"세금": float(tax_amount),
            "세후 순수익": net_before - float(tax_amount)}

def multi_year_summary(df, years, tax_by_year: dict[int, float] | None = None):
    tax_map = {int(k): float(v) for k, v in (tax_by_year or {}).items()}
    rows = [{"연도": str(y), **year_totals(df, y),
             **apply_tax(year_totals(df, y)["순수익"], tax_map.get(int(y), 0.0))}
            for y in years]
    return pd.DataFrame(rows + [_cumulative(rows)])   # 4년 누적행 포함
```

★ **핵심 구현:** I/O 없는 순수 함수 설계 → **Colab/Jupyter 에서 동일 로직 재사용 가능**. **세율 추정 → 실제 납부 금액 직접 입력**으로 전환 (세무사 고지액이 가장 정확). 연도별 dict 로 관리, 로컬 `tax_by_year.json` 영속화.

budget_app / mobile_ui.py

Mobile-First | Responsive CSS

```python
# 모바일 우선 전역 CSS 주입 + Plotly 모바일 설정
_GLOBAL_CSS = """
/* 기본 = 모바일 560px, 768px+ 에서 데스크탑 업그레이드 */
.main > div.block-container { max-width: 560px; padding: 1rem; }
@media (min-width: 768px){ .main > div.block-container { max-width: 760px; }}
.stButton > button { min-height: 48px; }                /* 엄지 터치 */
.stTextInput input, .stNumberInput input {
    font-size: 16px !important;                         /* iOS 자동줌 방지 */
}
"""
def mobile_chart_layout(height=300):
    return dict(height=height,
        legend=dict(orientation="h", y=-0.15),      # 범례 하단
        margin=dict(l=8, r=8, t=20, b=40))        # 좌우 여백 최소
```

★ **핵심 구현:** Streamlit **모바일 최적화 CSS 모듈 분리** — 사이드바 → 상단 탭바, 데이터 에디터 → 월 카드 폼, 48px 터치 타겟, iOS 자동줌 방지 16px 입력폰트. 스마트폰 홈화면 바로가기 등록으로 네이티브 앱 느낌.

Technology Stack Overview

Python 3.10+ | LangGraph | Claude API | Gemini API | Groq API | FastAPI | Flask | Streamlit | Telegram Bot API | SendGrid SMTP | Docker Compose | AWS EC2 | aiosqlite | Chart.js | Plotly | pandas | GitHub Actions CI/CD | WebSocket | Upbit REST API | 한국투자증권 Open API | Hunter.io API | SMTP / IMAP | DNS MX Lookup | RSS / feedparser | asyncio | WeasyPrint | uvicorn | Google Sheets API | gspread | google-auth | python-dotenv | Streamlit Community Cloud | Storage Adapter Pattern | Config Source Adapter | Dual-Write Durability | Twelve-Factor Config | Mobile-First CSS